

An End-to-End Agent for Kaggle Competitions

Benchmarked against two frozen yardsticks on 20 competitions

Report v6 (August 2026)

All printed numbers are real measurements as of 2026-08-18; TBD marks the four lanes still to finish and score.

Wei-Hao Huang

Draft compiled August 18, 2026

Abstract

We present **my-agent**, an end-to-end agent for machine-learning competitions, and the design decision at its centre: *where external knowledge enters the pipeline*. The agent runs the full loop — problem identification, EDA, feature engineering, modelling, evaluation, submission — with an LLM reasoning at every stage, Auto-ML tools invoked through the shell, a cross-competition experience library, and tree search as the optimisation loop. Measuring our own first design showed knowledge injected downstream at the blending stage worth only $\sim 10^{-5}$; the architecture therefore injects it upstream through a *problem-identification layer* that classifies task family, train–test window relation, split policy, and external-data need before any modelling, emitting typed operators into a whitelisted, leakage-guarded external-data module. Local validation repeatedly proved unreliable for between-agent claims in development, so every verdict is scored on the real leaderboard. Under that discipline the layer separates cleanly — firing on exactly the 2 of 18 baseline competitions that need external data, silent on the other 16; all six external-data arms across three firing-class competitions beat their baselines with paired significance; a with-layer/without-layer ablation passes the paired test on the benchmark’s canonical failure; and no local signal we tested selects the *form* the injection should take — the one selector surviving hindsight was killed by its first pre-registered prediction. We benchmark the agent against two frozen, methodologically distinct yardsticks — **AIDE** (LLM tree search) and the **NVIDIA Kaggle Agent** (kernel reproduction) — on 20 Kaggle competitions under a mechanically enforced fairness protocol, the my-agent lanes credential-free and transcript-audited, plus three mid-complexity competitions (NLP, physiological time series, medical imaging) where my-agent takes the three-way best on all three. The 16 completed lanes (one running, three queued as this draft compiles) were scored on the real leaderboard by the separate credentialed operator step on 2026-08-18: under the paired error model of Section 3, my-agent leads the standing with 15 wins and 5 losses in its decided duels (75% excluding ties, against 42% for AIDE and 38% for NVIDIA), taking the best private score on 10 of the 16; the remaining duels await the four unfinished lanes.

1 Introduction

Machine-learning competitions are *scorable tasks*: a submission receives a single measurable quality score on a hidden test set, and a public leaderboard records how it compares against thousands of human teams. Solving them end-to-end — data understanding, validation design, feature engineering, model selection, ensembling — has traditionally demanded expert labor and extensive trial and error. LLM-driven coding agents have recently begun to automate this loop: AIDE performs LLM-guided tree search over candidate solutions [2]; ERA couples an LLM rewriter with tree search and externally injected research ideas, outperforming strong baselines across 16 Kaggle Playground competitions [1]; and industrial agents skip search altogether by reproducing the strongest public solution to a given competition [3].

This report is about one such agent, **my-agent**, built over the course of this internship: a hybrid design in which an LLM reasons at every stage of the competition pipeline, Auto-ML libraries are invoked through the shell for what they do best, a cross-competition experience library supplies evidence-backed priors, and a tree search over candidate configurations serves as the optimisation loop (Figures 1 and 2).

The design question we ended up caring most about is *where external knowledge should enter the pipeline*. Our first answer was the conventional one — inject it downstream, among the candidates being blended — and measuring our own implementation showed that answer to be worth about $\sim 10^{-5}$. A failure-mode reading of all three agents studied here located their shared gap upstream, at problem-level judgment — “does this task need external data?”, “is the test set a future time window?” (summarized in Section 2.2). The architecture therefore injects knowledge upstream, through a *problem-identification layer*: a dossier stage that runs before EDA, classifies the task, and emits typed operators into a whitelisted, leakage-guarded external-data module (Section 2.2). That layer is evaluated under real-leaderboard discipline in Sections 5.1 and 5.2 — the scored evidence at the centre of this report.

Testing an architectural claim of this kind requires reference points we did not build and cannot tune, so all results are stated against two frozen, methodologically distinct yardsticks: **AIDE**, LLM tree search with no cross-competition memory, and the **NVIDIA Kaggle Agent**, which reproduces the highest-voted public kernel. All three run on 20 Kaggle competitions plus three mid-complexity competitions; because local estimates cannot be trusted to arbitrate between agents, every yardstick verdict is scored on the real leaderboard via late submission. my-agent’s benchmark lanes run under the frozen architecture in credential-free sandboxes (Section 3.1); 16 of 20 are complete as this draft compiles (one running, three queued), and the separate credentialed operator step scored those 16 on 2026-08-18, so the benchmark tables carry real my-agent leaderboard scores and paired verdicts for them, with the four unfinished lanes TBD (Sections 4.2 and 4.4).

Reading such comparisons at all requires an error model — top-two leaderboard gaps often sit at the fourth or fifth decimal place, and agent execution is itself stochastic — so every verdict in this report is adjudicated under the measurement methodology of Section 3.

What the report contains: the architecture and its injection layer (Section 2.2); the measurement methodology (Section 3); the 20-competition benchmark with my-agent’s local numbers and the yardsticks’ frozen leaderboard scores (Section 4.2); the injection layer’s controlled, leaderboard-scored experiments (Sections 5.1 and 5.2); and a mid-complexity three-way across NLP, physiological time series, and medical imaging where my-agent takes the best score on all three (Section 6).

2 Methods

2.1 my-agent: A Hybrid Kaggle Agent

The method is implemented as a Claude Code Skill (`kaggle-agent`). Its core design is a division of labor between LLM reasoning and Auto-ML tools: the LLM (Claude Code) handles problem understanding, validation-strategy design, creative feature engineering, result interpretation, and deciding the next step; LightGBM / XGBoost / CatBoost / Optuna — invoked by the LLM via the shell — handle model selection, hyperparameter optimization, and stacking/ensembling.

Execution proceeds through the pipeline of Figure 1: data ingestion → **Stage 0.5** problem dossier → EDA → feature engineering → modeling → evaluation → submission. The dossier stage decides what kind of problem the task is before any modeling (Section 2.2); it is evaluated in its own right, under the same paired-test discipline as everything else, in Sections 5.1 and 5.2. Modeling and evaluation run in one of two optimization modes:

- **Linear iteration protocol**: propose a hypothesis, run the experiment, record the result — used for the first pass and for small competitions.
- **Tree search (preferred)**: once the linear protocol has produced a baseline single model and at least one blend, the agent switches to tree search. Internal evaluation shows tree search winning 9 of 10

competitions with 1 tie and 0 losses.

Three cross-cutting mechanisms support the pipeline:

Cross-competition experience library (`knowledge/experience.md`): every insight must carry an evidence field (`competition`, `exp #N`, `score A` $\rightarrow$ `score B`); entries without a score delta are rejected. The agent queries the library before EDA and before modeling, and writes validated new insights back after experiments. This is the most fundamental design difference between my-agent and the two reference methods.

Experiment records. Every experiment’s parameters, features, split scheme, and scores are logged in structured form (`experiments.json`, `experiments_tree_v3.json`), making every number in this report traceable.

External-data layer : realizes the dossier’s typed operators as whitelisted, leakage-guarded external joins, each raced against the no-external baseline inside the search (Section 2.2).

Figure 1 summarizes the architecture and how the three mechanisms connect to the search loop.

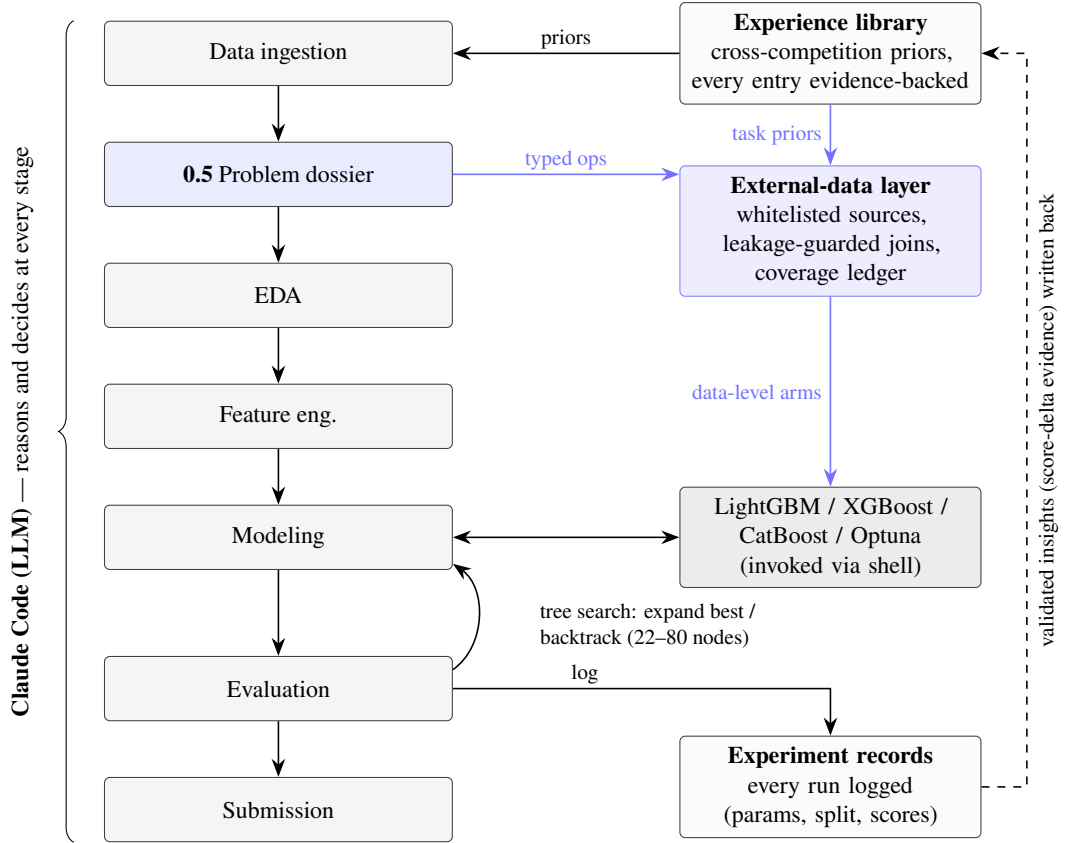


Figure 1: Architecture of my-agent. The LLM drives every stage; modeling and evaluation iterate via tree search, invoking Auto-ML tools through the shell, while the experience library and experiment records close the evidence loop. Blue: the injection layer (Section 2.2) — the Stage 0.5 dossier emits typed operators that drive the external-data layer, whose data-level arms race against the no-external baseline.

Tree-search protocol. Every run in this report uses the tree-search harness of Figure 2 (`tree_search/harness_v3.py`): ensembles are first-class node kinds with out-of-fold caching and weight search, and a budget phase machine exploits until all lineages plateau, then forces one exploration burst, then stops. An earlier downstream variant of the harness injected external ideas at the blend stage; no benchmark submission here used it, and its measured near-null effect is what motivated the upstream layer (Section 2.2).

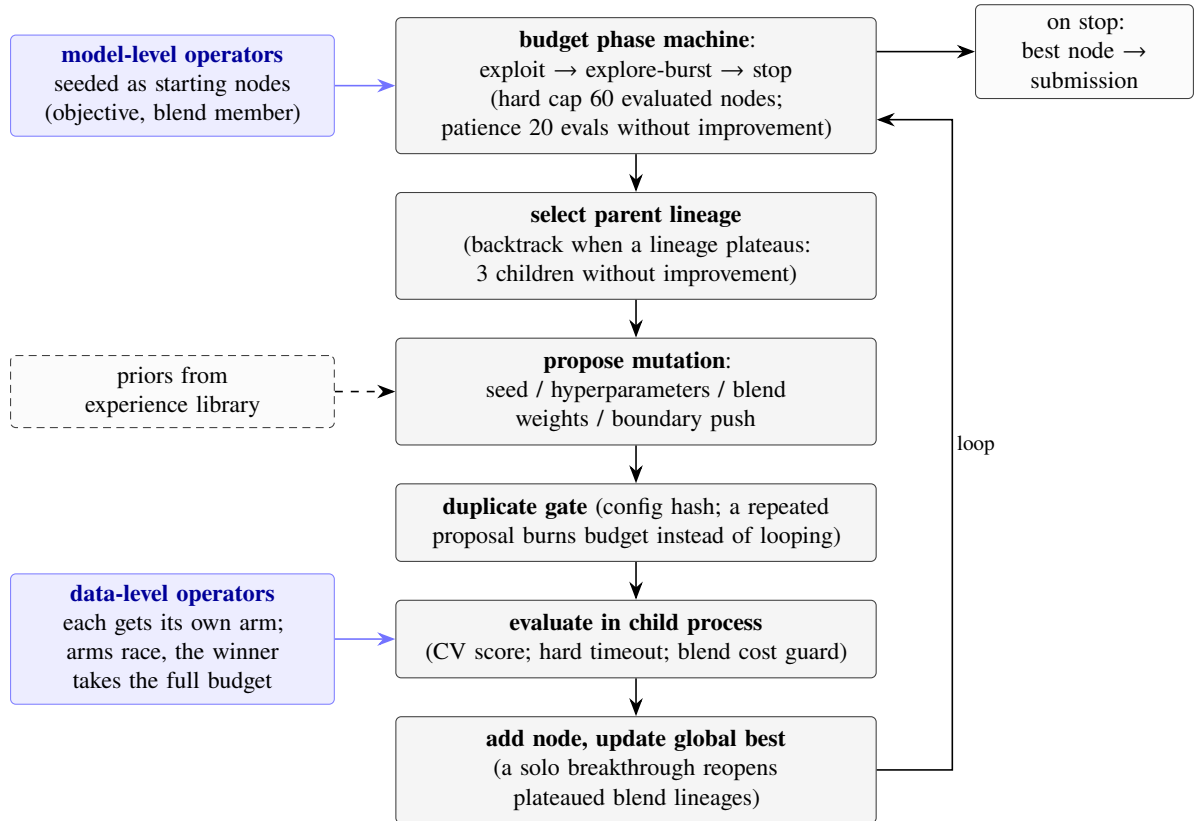


Figure 2: The tree-search loop used in all my-agent runs of this report (22–80 nodes per competition). Each node is a complete, executable solution scored by local cross-validation; lineages are abandoned on plateau, and the budget phase machine forces one exploration burst before stopping. Blue: the injection layer’s operator entry points (Section 2.2) — model-level operators seed starting nodes; data-level operators build separately raced arms.

2.2 The Injection Layer: Dossier, External Data, and the Typed Contract

The blue elements of Figures 1 and 2 form a single layer, and its placement carries the design argument. An earlier downstream variant had injected external ideas at the blend stage and measured a negligible effect (reweighting never displaced the convex-optimum champion in 8/8 rigorous competitions; pool expansion helped 2/8 by $\sim 10^{-5}$); the architecture moves the injection point upstream to problem identification, where a failure-mode reading of the three agents’ development runs locates their shared gap.

The failure-mode reading that motivated the layer. Reading the three agents’ development-run execution logs and champion code located their shared gap at problem-level judgment rather than tuning skill. AIDE can diagnose but has no mechanism that lets a diagnosis veto a score: on s3e19 it accidentally swapped its time-based validation for shuffled KFold mid-run, its metric instantly “improved” from 18.42 to 4.41 — an illusion a controlled split-only experiment measures at $4.5\times$ — and it tuned against the leaked number to the end, while my-agent quarantined the same split as “diagnostic only.” NVIDIA reproduced its source kernel’s skeleton on the same competition but missed the step that decides the outcome, and its vote-based selection signal measures popularity rather than quality — nothing in the method checks the reproduced score against the source’s claimed rank. And all three agents missed s3e19’s need for external data: at matched budget the per-country GDP join the leaderboard’s better half used is worth about 6 SMAPE of a ~ 44 -point gap (Section 5.2), the remainder a full-future-year extrapolation that defeats every method equally. The dossier layer below is the architectural answer to exactly this class of miss.

Stage 0.5 — problem dossier. Before EDA, the agent answers “what kind of problem is this?”: task family, train–test window relation, split policy, and external-data need, written to a structured `dossier.json`. Judgments are grounded in `knowledge/task_priors.md`, a library of task-level priors each carrying an evidence field from our own benchmark (e.g., the shuffled-KFold $4.5\times$ bias; the GDP-join CV gap on jan-2022). Reading competition-specific discussions or kernels is forbidden — the dossier must derive from the problem statement plus task-*type* knowledge only, keeping the method distinct from reproduction-based agents. The dossier is a prior, not a conclusion: EDA must verify its hypotheses and record a verdict.

External-data module. A whitelist of public sources (eleven World Bank indicators, ECB monthly FX, OWID COVID, holiday calendars, ISO tables), each fetched with recorded snapshots and joined through leakage-guarded as-of merges: an external value may reach a prediction row only if dated at or before the row’s period minus a declared lag; violations raise an error; numeric time columns must declare their unit; every join writes an audit log. The module was adversarially verified (21 reproduced findings, including silent-future-leak classes, all fixed with regression tests).

Search integration. Data-level dossier ideas spawn their own variant workspaces raced at small tree-search budget; model-level ideas enter the main tree as seed lineages. The winning lane receives the full search budget.

A typed contract between the two layers. The first version of this stack failed instructively: the dossier correctly proposed World Bank GDP for s3e19 *as a country-level scale covariate*, and the experience library already held the operational recipe, but the execution layer could express exactly one transformation — appending a column to the feature list. Since a gradient-boosted tree cannot extrapolate a feature beyond its training range, the semantics of “scale covariate” evaporated between the layers with nothing raising an error. The fix is a closed vocabulary both layers read (`knowledge/injection_operators.md`): ten typed operators, each with a parameter schema, the evidence justifying it, and explicit conditions under which it must *not* be used. The dossier may emit only these, each carrying its rationale and source prior; anything the vocabulary cannot express is written to `not_recorded` rather than into prose the execution

layer will ignore. The dispatcher writes a coverage ledger — proposed, realized, unrealized-with-reason — so an unimplemented capability leaves a trace instead of resembling success. Two automatic pre-conditions guard the level-covariate operators: proportionality and covariate volatility. Conformance is machine-checked on both sides (Appendix A.5).

2.3 Two Reference Methods (Yardsticks)

AIDE (open-source agent, gen-1) performs LLM-driven tree search: at each step it writes a complete solution, executes it, reads the error or score, and mutates accordingly. It carries no cross-competition memory — every competition starts from zero.

NVIDIA Kaggle Agent (skill) reproduces the strongest public kernel: it researches the competition context, selects the highest-*voted* solution-type kernel, ports it faithfully, and retrains on local data.

Both reference methods are frozen — no improvements of any kind, even where we found weaknesses (Section 2.2; the NVIDIA skill even ships a score-based selection tool we deliberately left disabled). Improving the yardsticks mid-study would make earlier and later data incomparable and would quietly turn the benchmark into an optimization of the reference methods.

3 Measurement Methodology

Two pieces of apparatus are prerequisites for every three-agent comparison and every real-leaderboard verdict in this report, and neither is a finding in its own right: a fairness and execution-discipline protocol that makes a score attributable to a method rather than to an accident of the shared machine, and an error model that says which score differences are readable at all. We state both here, once, so that the experimental sections can report results without re-deriving the conditions under which they mean anything. The offline-graded mid-complexity competitions of Section 6 are the one exception to the error model’s reach, noted there.

3.1 Fairness Protocol and Execution Discipline

The three agents share one machine, so four isolation conditions are defined and mechanically enforced rather than left to convention (Table 1): if the agents could read each other’s files, or run concurrently on a contended machine, no verdict below would be attributable to method. Implementation details and the failure record are in the companion engineering log.

Table 1: The four isolation conditions and what enforces each.

Condition	Rule	Enforcement
Data	Each side reads only its own root, containing files validated against the official Kaggle manifest	<code>build_isolated_roots.py</code>
Workspace	The three output directories are fully separated; each side is scored independently	Directory structure
Instruction	Prompts may carry environment facts and metric thresholds only — no modelling recipes, no other side’s scores, no workaround discovered by another side	Manual protocol + incident log
Resource	Only one lane executes at any moment	<code>lane_lock.sh</code> mutex

A fifth rule governs the experimenter rather than the agents: when the environment is defective, fix the environment rather than hint to the affected side. Fixing removes the confound; hinting compensates one side and silently changes what is being compared.

Benchmark execution discipline. For my-agent’s 20 benchmark lanes the protocol is tightened further. A benchmark manifest (`docs/rerun_manifest.json`) fixes exactly one workspace, one canonical evaluator, and one search driver per competition, after a development audit found workspaces accumulating up to 13 candidate evaluators. Each lane executes in an isolated root whose data files are validated against the official Kaggle manifest, holds *no credentials* (the Kaggle token is stripped from the environment, so a lane can neither consult the leaderboard nor submit), and sees only its own workspace. A *per-lane transcript audit* then reads every tool call the lane made — detectors for network access, raw opens of the shared knowledge library that bypass the query interface’s self-citation exclusion, and results of sibling benchmark competitions appearing without a sanctioned source — and the launcher halts the whole queue after two consecutive audit failures, on the reasoning that two in a row indicts the pipeline rather than one lane. Section 4.3 reports the audit gate exercised in both directions on the actual run.

3.2 The Paired Significance Test

Why an error model is needed at all. In the development-lane leaderboard measurements this apparatus was built on, several top-two gaps sat at the fourth or fifth decimal place (s5e10: 0.00003; s6e2: 0.00000). Do such gaps exceed measurement error? If not, the “verdict” is noise — and a report that declares winners at that scale is reporting its own noise back to itself.

How — the error scale comes from the data itself, not from assumptions. Kaggle randomly splits each test set into two disjoint subsets, public and private, and one submission receives one score on each. These are two independent estimates of the same model, so their difference is that competition’s metric sampling noise.

A failed first attempt, worth recording. We initially used a single agent’s own public–private gap as the threshold, and 8 of 10 competitions came out as ties. The reason is that this gap is mostly *shared* across the three agents — which rows land in which subset shifts everyone together. On s5e10 (Table 2) the common shift is 0.00025 against mutual scatter of only 0.00002: using the common shift as the threshold inflates the true noise more than tenfold — measuring a paired question with an unpaired scale.

The correct construction. Measure how much the *gap between two agents* moves across the two subsets — the common drift cancels in the subtraction. Writing d_{pub} and d_{priv} for the between-agent score difference on the public and private subsets, a verdict requires both:

$$\text{sign}(d_{\text{priv}}) = \text{sign}(d_{\text{pub}}) \quad (\text{the ordering replicates on disjoint samples}) \quad (1)$$

$$|d_{\text{priv}}| > |d_{\text{priv}} - d_{\text{pub}}| \quad (\text{the gap exceeds its own movement}) \quad (2)$$

Statistical properties of the decision rule. Algebraically, condition (2) is equivalent to $d_{\text{pub}} (2d_{\text{priv}} - d_{\text{pub}}) > 0$ and therefore *implies* condition (1); we state both because they diagnose different failures. The rule is a conservative screen rather than a calibrated hypothesis test, but its false-verdict rate under a null model is computable. Suppose two agents are truly equivalent on a competition, so that d_{pub} and d_{priv} are independent zero-mean Gaussian noise with $\sigma_{\text{pub}} = k \sigma_{\text{priv}}$. The probability of a spurious verdict (in either direction) is

$$P_{\text{FP}}(k) = \frac{1}{2} - \frac{\arctan(k/2)}{\pi}, \quad (3)$$

which gives 35.2% for equal variances ($k = 1$) and 25.0% for the 20%/80% public/private splits common in these competitions (variance scales inversely with subset size, so $k \approx 2$); condition (1) alone caps it at 50% distribution-free. This is the rule that adjudicates every leaderboard verdict in this report, including the benchmark duels of Section 4.4.

Table 2: Empirical evidence for common drift (excerpt; development-lane submissions). Drift = private minus public score of the same submission. In seven competitions the mutual scatter is less than half the common shift.

Competition	my-agent	NVIDIA	AIDE	Common shift	Mutual scatter
s5e10	+0.00025	+0.00026	+0.00024	+0.00025	0.00002
s6e2	+0.00149	+0.00160	+0.00152	+0.00154	0.00011
s3e11	+0.00063	+0.00065	+0.00056	+0.00061	0.00009
s3e7	−0.00859	−0.00772	−0.00825	−0.00819	0.00087

The second error term: between-run agent variance. The paired test covers test-set sampling noise only; agent execution is itself stochastic. On the four competitions with the benchmark’s tightest development-lane duels, my-agent ran twice fresh under identical conditions (fresh workspace, knowledge library pinned per competition to the last commit before any information about that competition entered it), and AIDE’s pinned re-run pairs against its frozen baseline. Private-leaderboard spreads:

	my-agent	AIDE	development-lane duel gaps
s4e11	0.00021	0.00027	0.00013–0.00033
s3e16	0.00460	0.00056	0.00142–0.00365
s5e10	0.00000	0.00003	0.00003–0.00012
s6e2	0.00000	0.00020	0.00000–0.00008

Three readings, condensed. The variance is substantially a property of the *task*, not the agent — both agents agree s3e16 is the noisy competition by an order of magnitude — so error bars must be per-competition, not one global number. Several fourth-decimal development-lane gaps sit at or below a measured wobble (s5e10’s 0.00003 gap exactly equals AIDE’s wobble there), so the combined error model treats gaps at that scale as undecidable — a floor the benchmark verdicts of Section 4.4 apply. And the pipeline’s endpoint is far more stable than its route: my-agent’s two runs took visibly different paths on every competition yet landed within 0.0002 everywhere except the task both agents find noisy. NVIDIA’s variance term remains unmeasured (Section 7.5).

4 The Three-Way Benchmark

4.1 Overview of Experimental Design

Evaluation set. 20 tabular competitions — 14 from the Kaggle Playground Series, plus afsis-soil-properties (2014, spectral regression), conway-s-reverse-game-of-life (2014, structured multi-output), cat-in-the-dat (2019, purely categorical features), and tabular-playground aug-2022, jan-2022, and sep-2022 (Table 3). Every competition provides both a public and a private test-subset score — the inclusion criterion required by the significance test (Section 3.2). The three agents also ran 4 competitions for which private scores cannot be obtained (late submission closed, rolling leaderboards); these are excluded. The set grew from a frozen 18-competition baseline to 20 when the two firing-class competitions (s5e1, tps-sep-2022) joined on 2026-07-31 after all three lanes ran them under benchmark conditions — a post-hoc expansion disclosed as a limitation (Section 7.4).

Execution conditions. The three agents ran serially under the mutex of Section 3.1. AIDE: 20 steps, its own default budget (conway stopped at 16 due to the 4-hour cap). NVIDIA: reproduced public kernels per its method. my-agent: the full six-stage pipeline including tree search (22–80 nodes per competition; Appendix A.6), each lane under the execution discipline of Section 3.1.

4.2 Three-Way Capability Comparison (Raw Scores)

Goal and procedure. Measure the absolute performance of the three methods on the same set of competitions. Each side independently produces a submission, scored on Kaggle via late submission for public and private scores and a percentile rank (PR). The AIDE and NVIDIA columns of Table 4 are those frozen leaderboard scores. my-agent’s lanes hold no credentials, so nothing in a lane ever touches the leaderboard: each lane freezes a `submission.csv`, and scoring is a separate credentialed operator step (`score_lane_submissions.py`; dry-run by default, real submission behind an explicit flag), after which `build_rerun_three_way.py` places the my-agent column against the yardsticks under the paired test of Section 3.2. That operator step ran on 2026-08-18 for the 16 completed lanes, and their real public/private scores fill the LB column of Table 4; the four unfinished lanes and all my-agent percentile ranks (which require a leaderboard-snapshot pull) remain TBD. The local CV column reports each lane’s own honest out-of-fold estimate, which must not be read as a leaderboard proxy (s3e19’s pooled OOF SMAPE of 6.71 sits against a realized lane score of 49.41). Per-competition specifications for every agent are attached (Appendix A).

Table 3: Competition descriptions. Tasks fall into two families: classification (binary, ordinal; 7 competitions) and prediction/regression (incl. time series and multi-target; 13 competitions).

#	Competition	Task type	Train rows $\times$ cols
1	s3e1	Regression	$37,137 \times 10$
2	s3e3	Classification (binary)	$1,677 \times 35$
3	s3e5	Classification (ordinal)	$2,056 \times 13$
4	s3e7	Classification (binary)	$42,100 \times 19$
5	s3e9	Regression	$5,407 \times 10$
6	s3e11	Regression	$360,336 \times 17$
7	s3e14	Regression	$15,289 \times 18$
8	s3e16	Regression	$74,051 \times 10$
9	s3e19	Regression (time series)	$136,950 \times 6$
10	s4e11	Classification (binary)	$140,700 \times 20$
11	s5e10	Regression	$517,754 \times 14$
12	s6e1	Regression	$630,000 \times 13$
13	s6e2	Classification (binary)	$630,000 \times 15$
14	afsis-soil-properties	Regression (multi-target)	$1,157 \times 3,600$
15	cat-in-the-dat	Classification (binary)	$300,000 \times 25$
16	conway-s-reverse-game-of-life	Structured multi-output	$50,000 \times 802$
17	tps-aug-2022	Classification (binary)	$26,570 \times 26$
18	tps-jan-2022	Regression (time series)	$26,298 \times 6$
19	s5e1	Regression (time series)	$230,130 \times 6$
20	tps-sep-2022	Regression (time series)	$70,128 \times 6$

PR is the share of that competition’s final private standings the submission would beat (a would-be rank r among N teams gives $\text{PR} = 100 (N - r)/N$, against the frozen final leaderboard) — an *absolute* capability reference (“where would this agent have placed had it entered?”), separate from the relative ordering among the three.

4.3 Execution and Audit of the Benchmark Run

The queue ran 2026-08-14 through 08-18 under operator approval, halting three times — twice on environment faults at lane boundaries (which the launcher distinguishes from a lane result by exit timing) and once on its two-consecutive-audit-failures rule — with 16 of 20 lanes complete and audit-accepted, one (s6e2) running on its second attempt, and three queued as this draft compiles. The audit gate was

Table 4: The 20-competition benchmark, state as of 2026-08-18. All leaderboard cells are public/private scores; **bold** marks the best *private* score of the three (raw, direction-aware; the paired-test verdicts, which this raw reading does not replace, are in Table 5). my-agent’s local CV (OOF) is each lane’s own honest out-of-fold estimate, not leaderboard-validated; its LB column comes from the credentialed operator scoring step of 2026-08-18, and its percentile ranks await a leaderboard-snapshot pull. NVIDIA and AIDE cells are frozen leaderboard scores with percentile rank (PR) in parentheses. Lane status: 16 of 20 lanes complete and scored (~19 hours of lane time); s6e2 running; tps-aug/jan/sep-2022 queued (TBD rows, not bolded).

#	Competition	Metric	Local CV (OOF)	my-agent LB (pub/priv)	NVIDIA LB (PR)	AIDE LB (PR)
1	s3e1	RMSE ↓	0.548862	0.55715/ 0.55275	0.55332/0.55550 (93.2)	0.55899/0.55552 (81.0)
2	s3e3	ROC-AUC ↑	0.841605	0.89449/0.87157	0.93526/ 0.89537 (66.8)	0.88484/0.86863 (26.4)
3	s3e5	QWK ↑	0.571980	0.57041/0.57223	0.57515/0.56974 (75.7)	0.59362/ 0.58138 (79.8)
4	s3e7	ROC-AUC ↑	0.901769	0.91127/0.90441	0.91230/ 0.90458 (78.1)	0.90934/0.90109 (67.1)
5	s3e9	RMSE ↓	12.082282	11.84750/12.33192	11.83399/ 12.22913 (59.1)	11.86657/12.30667 (44.5)
6	s3e11 ^a	RMSLE ↓	0.292886	0.29309/ 0.29368	0.29559/0.29624 (66.6)	0.29389/0.29445 (73.3)
7	s3e14	MAE ↓	340.95936	343.43434/333.18611	338.36524/ 330.71616 (89.2)	342.44918/332.31556 (71.2)
8	s3e16	MAE ↓	1.334958	1.33738/ 1.33822	1.33566/1.34001 (96.4)	1.33920/1.34224 (91.4)
9	s3e19 ^b	SMAPE ↓	6.70585	49.11872/49.40878	47.52145/ 48.26558 (41.4)	48.39715/48.34131 (40.1)
10	s4e11	Accuracy ↑	0.940810	0.94195/ 0.94107	0.94136/0.94076 (52.8)	0.94184/0.94056 (62.1)
11	s5e10 ^c	RMSE ↓	0.055971	0.05554/ 0.05576	0.05562/0.05588 (59.4)	0.05555/0.05579 (78.9)
12	s6e1 ^d	Error ↓	0.787400 ^d	8.68844/ 8.71516	8.70480/8.73239 (69.5)	8.69449/8.72191 (74.8)
13	s6e2 ^e	ROC-AUC ↑	TBD	TBD	0.95348/0.95508 (63.0)	0.95364/0.95516 (76.5)
14	afsis	MCRMSE ↓	0.470503	0.41902/ 0.48917	0.61276/0.69855 (18.8)	0.45636/0.49861 (45.6)
15	cat-in-the-dat	ROC-AUC ↑	0.803272	0.80800/ 0.80243	0.77641/0.77084 (32.6)	0.80790/0.80217 (70.3)
16	conway ^g	MAE ↓	0.106688	0.10313/ 0.10414	0.11094/0.11189 (96.5)	0.10960/0.11065 (97.9)
17	tps-aug-2022	ROC-AUC ↑	TBD	TBD	0.57920/0.58730 (43.0)	0.58479/0.59106 (60.5)
18	tps-jan-2022	SMAPE ↓	TBD	TBD	4.43958/4.63651 (76.4)	10.11598/10.66638 (28.9)
19	s5e1	MAPE ↓	5.84% ^f	0.09076/ 0.08401	0.05726/0.14594 (69.3)	0.12573/0.17917 (48.1)
20	tps-sep-2022	SMAPE ↓	TBD	TBD	4.77458/5.48351 (80.3)	5.78998/9.72095 (48.3)
Median PR (range)				pending snapshot	68.0 (18.8–96.5)	68.7 (26.4–97.9)

^a First attempt exited without a submission; completed 08-17. ^b Clean re-run 08-17; first attempt discarded by the transcript audit (Section 4.3). ^c Audit incident resolved, score accepted (Section 4.3). ^d Completed and audit-accepted 08-17. The lane’s local metric is R^2 (maximize; pooled OOF 0.787400), not the leaderboard’s error metric, so this cell is not comparable in scale or direction to the LB columns. ^e Second attempt, launched 08-18; the first attempt exited without a submission. ^f The lane’s cross-arm blend weights and global scale are fitted on the same OOF rows they are scored on, so its 5.84% is optimistic relative to a held-out estimate — the lane’s own summary records this caveat. ^g The scoring tool’s structural validator false-blocked conway’s file (sample-file naming); the operator submitted the identical file (same SHA-256) manually via the Kaggle CLI.

exercised in both directions, most sharply on **s3e19**: the 08-14 lane opened the raw knowledge library four times and issued a genuine socket-opening command, and the audit discarded it. The clean 08-17 re-run scores *worse* on local CV — pooled OOF SMAPE 6.706 against the tainted attempt’s 5.740 — consistent with the contamination advantage being real and the audit pricing it correctly; a benchmark that discards a better-looking number and keeps the honest one has evidence its audit does something. The other two incidents were resolved by reading transcripts rather than trusting either the lane or the auditor: the network detector fired on an `ls` of a directory named `kaggle-safe-submit` (a manual scan of all 118 tool calls found zero network commands; accepted under a recorded operator waiver), and on **s5e10** the flagged raw library opens all occurred strictly *after* the submission was written and validated, so the score stands — the genuine violation there, a hand-edit of the shared knowledge library after finalisation, was surgically reverted before any later lane ran. After the audit, the credentialed operator step submitted the 16 frozen submissions on 2026-08-18; their scores enter Table 4 and Section 4.4.

4.4 Standing of the Three Agents Under the Paired Test

The benchmark’s headline reading is the full duel matrix: 20 competitions $\times$ 3 pairings = 60 duels, each adjudicated by the rule of Section 3.2 plus the per-competition variance floors. With the 16 completed lanes scored, 52 of the 60 duels are adjudicated (the yardstick-vs-yardstick duels need no my-agent score); the 8 my-agent duels on the four unfinished lanes are pending. Of the 52, 34 produce a verdict and 18 are undecidable — the class the error model exists to catch, including a live replication of the development-lane **s5e1** case (the public half of that test split favours NVIDIA, the private half favours my-agent, so the duel is a tie by construction) and two duels killed by the variance floors, both wins my-agent would otherwise have claimed: **s3e16**, and **s5e10**, where the private gap of 0.00003 exactly equals AIDE’s measured wobble — the case Section 3.2 flagged in advance. Table 5 gives the standing. my-agent leads outright: it wins 15 of its 20 decided duels (75% excluding ties), takes the best private score on 10 of the 16 scored competitions, and its rerun submission beats its own development-lane private score on 11 of 16.

Table 5: Standing under the paired test with per-competition variance floors, as of the 2026-08-18 scoring step: 52 of 60 duels adjudicated (34 decided, 18 undecidable), 8 pending the four unfinished lanes. Win rate excludes ties.

Method	Wins	Losses	Win rate (excl. ties)
my-agent	15	5	75%
AIDE	10	14	42%
NVIDIA	9	15	38%

5 The Injection Layer: Controlled Experiments

Acting on the diagnosis of Section 2.2 — problem-level judgment is the shared gap of all three agents — the experiments here evaluate the injection layer of Section 2.2 under the same leaderboard discipline as the benchmark: real submissions, judged by the paired test of Section 3.2.

5.1 Problem-Identification Sweep (Dossier Layer)

Design. The dossier layer’s generality claim is “behaves correctly across the task distribution”: fire on tasks that genuinely need external data (sensitivity), stay silent on those that do not (specificity). We ran the dossier stage on all 18 original benchmark competitions — LLM reasoning over the problem statement, data schema, and task priors only; minutes per competition; no training.

Result: perfect separation. The layer fires on exactly the two country-panel time-series competitions — s3e19 and jan-2022, the two whose accepted solutions are known to require external covariates — and reports “no / unlikely” on all 16 iid-tabular, categorical, spectral, and structured-output competitions: sensitivity 2/2, specificity 16/16. (Two caveats keep this honest. The sensitivity half is partly in-sample: the task priors the dossier reads were distilled from this benchmark’s own failure analysis of exactly these two competitions, so the 16/16 silence is the informative half, and 2/2 carries a 95% CI reaching down to 16%. And the two firing-class competitions added to the baseline later also fire, but they were *selected* as firing-class, so the sweep claim rests on the original 18.) Two secondary behaviors: on s3e19 the dossier surfaced a genuine conflict — a conservative setup-time `config.yaml` flag said external data was not allowed while the task prior’s evidence implied otherwise — and flagged it for verification rather than guessing (the official rules, Section 7.C, settle it in favor of permission); on the out-of-scope SIIM imaging competition it answers “no”, correct within the current whitelist, whose source classes do not yet include prior-competition archives.

5.2 What the Layer Delivers on the Real Leaderboard

Design. s3e19 is the benchmark’s canonical failure, so it is the causal test bed; tps-sep-2022 (held-out replication, dossier written in advance) and s5e1 (pre-registered test) complete the firing class. Arms are built so the operator’s added columns are the *only* difference from the baseline (verified digit-identical otherwise), and every arm is submitted for real public and private scores, judged by the paired test of Section 3.2. Full arm tables and probe-level detail are in the companion case-studies document.

Effect. Across the three competitions, **all six external-data arms beat their no-external baselines, every one paired-significant.** In matched configuration the feature join is worth 6.01 private SMAPE on s3e19 and 2.09 on the held-out sep-2022; at a full 46-node search budget per arm both external arms still beat the searched baseline; and on s5e1 the winning form takes a 20% relative private-MAPE margin (0.12417 vs. 0.15751), the largest effect in the class. On the canonical failure itself, the with-layer/without-layer ablation — the full pipeline against a searched champion from the identical pipeline with the layer disabled — passes the paired test (private 48.243 vs. 50.455; $d_{\text{pub}}+3.389$, $d_{\text{priv}}+2.212$, movement 1.177) and places the with-layer champion numerically ahead of both frozen yardsticks (48.26558 and 48.34131, Table 4).

The operating rule the experiments imposed. One negative result ships with the layer: no local signal we tested selects which *form* the injection should take. Cross-validation, a held-out-year protocol, and a proportionality diagnostic each misrank the form verdict (0-for-3); the diagnostic — the one selector that survived hindsight — was put to a pre-registered test on s5e1, its prediction committed to version control before any model was fitted, and its named falsifier fired. The layer therefore *races both forms inside the search, always*, and the one surviving post-hoc candidate (horizon length) is committed as the H-HORIZON pre-registration for the next firing-class competition (Section 7.5).

What the judgment layer proposes, and what actually consumes it. Across the 20 dossiers the layer proposes 241 ideas. Counting those whose operator family exists gives 73%; tracing which are actually *consumed* gives only 5% at first measurement, because the three operators supplying most of that headline were emitted into a key that no search driver read, while the dispatcher recorded them as realized (now 27%, after wiring a consumer and moving the target-free encodings into the data layer). A produced artifact with no consumer is indistinguishable from a working one in every log, score table and coverage metric — and this instance occurred inside the very mechanism built to prevent silent truncation. **An architecture claim needs a consumer trace, not a vocabulary count.** Per-regime tables and audit scripts are in Appendix A.5.

6 Generalization: Mid-Complexity Three-Way

Design. The three agents run, each under its frozen method, on three competitions outside the tabular Playground family: us-patent-phrasing-to-phrasing-matching (NLP, text pairing), ventilator-pressure-prediction (physiological time series), and SIIM-ISIC melanoma (dermoscopic imaging) — machine fully dedicated per lane, 4–8h budgets, same isolation protocol. None of the three my-agent lanes ran the injection layer (Section 7.4) or tree search: a node here is a full deep-learning fine-tune (30–90 minutes, versus seconds for a tabular GBDT refit), so the prescribed 22–80 node budget would cost 15–100+ GPU-hours against the 4–8h available, and all three lanes took the skill’s documented fallback for exactly this condition — one linear pass to a validated single model plus a blend, with the high-leverage moves (backbone choice, seed bagging, blend weights) taken directly.

Scoring. The mid-complexity competitions use MLE-bench [4] prepared splits and are graded offline against the held-out private split (us-patent is a code competition and ventilator/SIIM live test sets differ from the prepared splits, so Kaggle late submission is structurally unavailable); medal thresholds and the above-median flag come from the frozen leaderboard snapshot.

Table 6: Mid-complexity three-way, offline-graded against the held-out private split. Bold = three-way best; † = above the frozen leaderboard’s median.

Competition	Metric	my-agent	NVIDIA	AIDE
us-patent	Pearson r ↑	0.86658 † (silver)	0.86160† (bronze)	0.65821
ventilator	MAE ↓	0.16525	0.40093	0.34591
SIIM-ISIC	AUC ↑	0.93529 †	0.89822	0.88579

Results. my-agent takes the three-way best on all three and the only above-median score on SIIM — a clean sweep across three unrelated task families, so the result is about pipeline discipline rather than tabular-specific tuning. Offline grading yields one score per submission, so the paired public/private test of Section 3.2 does not apply here.

7 Discussion

7.1 Contributions

- 1. An end-to-end competition agent whose knowledge enters at problem identification rather than at blending.** The layer and its controlled, leaderboard-scored evidence are Sections 2.2 and 5.1–5.2: clean 2-of-18 separation in the dossier sweep, six of six external-data arms paired-significant, the with/without ablation significant on the benchmark’s canonical failure — and the sharp negative result that no local signal selects the injection’s *form*, the one selector surviving hindsight killed by its own pre-registered falsifier.
- 2. The agent’s placement against two frozen, methodologically distinct yardsticks, with the adjudication rule committed before the scores existed.** 16 of the 20 benchmark lanes are complete and scored on the real leaderboard (2026-08-18 operator step): my-agent leads the paired standing — 15 wins, 5 losses, 75% excluding ties (Section 4.4) — with 8 duels pending the four unfinished lanes; also scored are the mid-complexity sweep (three-way best on all three) and every controlled arm of the injection study (Section 5.2).
- 3. A caution that constrains how any such claim may be stated.** A ranking is a property of method $\times$ evaluation set — in development runs the winner flipped between two reasonable competition sets, and

on s5e1 between the two halves of a single test split (Section 7.3). Demonstrated rather than asserted, it applies to our own agent first.

The measurement machinery itself — the paired test, the variance measurement, and the audited, credential-free execution discipline of Section 3 — is infrastructure rather than a contribution, and is packaged for reuse by other agent benchmarks.

7.2 Comparing the Three Methods

The differences lie not in tuning skill but in where knowledge comes from and how the next step is decided (Table 7).

Table 7: Design comparison of the three methods.

Aspect	my-agent	AIDE	NVIDIA
Knowledge source	Cross-competition experience library (accumulating)	None (from zero each time)	Public kernels fetched fresh each time
Search strategy	Six-stage pipeline + tree search (22–80 nodes)	Tree-shaped trial and error, 20 steps	No search; faithful reproduction
Basis for next step	Experience-library priors + OOF diagnostics	Previous step’s error or score	Highest-voted kernel
Validation rigor	Split chosen by task type; diagnostics tagged and excluded	No concept of split legality	Inherits the kernel’s split
External data	Injection layer (typed operators, whitelisted sources)	Never considered	Inherited from the kernel

NVIDIA wins where a mature, high-quality public solution exists — sep-2022’s 277-vote Ridge baseline, which reproduction inherited for its cleanest development-run win, is the textbook case — and loses where public kernels are stale or tutorial-grade or where its vote signal misleads (Section 2.2). AIDE wins on search depth with no external knowledge at all, and loses where a diagnosis needs to veto a score: it can produce a correct diagnosis and still select the optimistic node (Section 2.2). my-agent’s edge is validation rigor and accumulated knowledge — it quarantined the same shuffled split AIDE submitted — while its measured bottleneck was judgment at the problem-identification level: exactly the gap the injection layer was built to close, and the reason the controlled experiments of Sections 5.1 and 5.2 carry the report’s thesis. What remains open within that layer is form selection (0-for-3) and the unmeasured value of the dossier on competitions where it stays silent. The shared limitation: on s3e19 all three development runs landed in the lower mode of a bimodal leaderboard, none recognizing the need for GDP data — a join worth 6.0 SMAPE, while the dominant shortfall is a one-year extrapolation no method in the comparison addresses.

7.3 A Ranking Is Not a Property of the Method, but of Method × Evaluation Set

The most important methodological lesson of the development runs: same agents, same test — swap the competition set and the paired-test winner flipped between the 14-competition Playground subset and the broader 20-competition set (those standings belong to superseded development lanes and are not reprinted here; the mechanism is a property of the methods and survives). The mechanism: reproduction dominates where a mature public solution exists, autonomous methods dominate on obscure, build-your-own-pipeline tasks, and relative verdicts binarize many near-tie gaps, so the paired win rate is highly sensitive to sample composition even while absolute position (median PR) stays stable. The s5e1 duel of Section 4.4 reproduces the caution inside a single competition: the public and private halves of one test split order two agents oppositely. An agent benchmark’s conclusion must therefore be stated together with the composition of its evaluation set, and should report both an absolute metric (PR) and a relative one (paired win rate).

7.4 Limitations of the Experimental Design

- **The significance test covers only test-set sampling noise**, not between-run agent variance. That variance is measured for my-agent and AIDE on four competitions (Section 3.2) and enters the combined error model as a per-competition floor; NVIDIA’s term remains unmeasured, and widening that sample is the cheapest way to harden every verdict the benchmark will produce.
- **The duels are not independent trials**. Each competition contributes three duels computed from six shared scores, so the effective sample size is closer to 20 competitions than to 60 duels and interval estimates must use competition-level cluster bootstrap. A further asymmetry touches every NVIDIA duel: reproduced kernels were tuned by their authors against the realized public leaderboard (degradation extreme +155%, s5e1), so the paired test’s exchangeability premise is weakest exactly where the reproduction method is strongest.
- **Budgets are per-method conventions, not compute-matched**. AIDE runs its default 20 steps, my-agent 22–80 evaluated nodes plus a linear pass, NVIDIA minutes of reproduction; each method runs “as shipped,” so the comparison is methods-with-their-budgets, not methods at equal compute.
- **The sample is small, the ranking is sensitive to its composition, and part of it was added post hoc**. With 20 competitions and 60 duels, adding or removing a single competition can move any win rate by one to two percentage points (Section 7.3); the two firing-class competitions joined after their leaderboards had been consulted for the injection experiments, and in the development runs that addition moved every rate — so the 18-vs-20 composition is disclosed rather than hidden.
- **The experience-library asymmetry is disclosed, not eliminated**: my-agent carries cross-competition memory; the other two cold-start.
- **The mid-complexity lanes ran without the injection layer and without tree search** (Section 6), and their offline grading yields one score per submission, outside the paired test’s reach; their sweep is therefore a raw-score result, not a paired-verdict one.

7.5 Open Problems

Ordered by how much they would change this report’s conclusions.

Operator form cannot be selected from any local signal we have tested — 0-for-3, with one selector killed by pre-registration. Section 5.2 states the rule; the probe-level misrankings are in the companion case-studies document, and the split form verdict means no constant answer exists to fall back on. The one surviving candidate selector, horizon length, is post-hoc; testing it requires pre-registering it on the next firing-class competition before its leaderboard is consulted (the binding protocol is committed as the H-HORIZON pre-registration). This remains the sharpest open question here, because the form choice was worth more than the entire search budget on one competition and a 20% relative margin on another.

The firing class is small. The dossier fires on 2 of 18 baseline competitions, so the controlled evidence rests on two clean test points plus one held-out replication. Extending the class is worth more than re-running the 16 competitions where the layer stays silent: a re-run there changes nothing the layer does and instead re-rolls 48 of the 60 duels with the run-to-run variance measured in Section 3.2.

The experience library’s residual weak link is retrieval quality. The enforcement half is closed: the current architecture makes the library query binding — an experiment record without a `library_query` trace is invalid, and the audit tool checks this mechanically — after a development episode in which we re-derived a result the library already held. What remains open is quality, not enforcement: a neighbouring entry over-claimed and had to be corrected against leaderboard evidence, and no mechanism yet scores retrieved priors for applicability.

Smaller items. Whitelisting further source classes (prior-competition archives, public text corpora), and a direct ablation of the experience library.

8 Conclusion

This study set out to build an end-to-end competition agent and to find out honestly what its architecture is worth. The architectural answer is placement: downstream injection measured near-null, so knowledge enters at problem identification, and the layer’s controlled, leaderboard-scored evidence carries the claim (Section 5.2) — together with the negative result that no local signal selects the injection’s form, a failure we count as the method working, since a benchmark that can kill its own hypotheses is the thing this study set out to build. The benchmark verdict is partial but real: the 16 completed, transcript-audited lanes were scored on the real leaderboard on 2026-08-18 and adjudicated by the error model committed in advance (Section 3) — my-agent leads with 15 wins to 5 losses (75% excluding ties), 18 duels undecidable at the measured noise scale, 8 pending the four unfinished lanes — discipline the development runs earned, where the winner flipped with the evaluation set and the audit discarded a contaminated lane whose tainted score looked better than its clean replacement’s. The arc of the internship is a single loop closed: build the agent, benchmark it honestly, read its failures, fix the layer they identify, and re-verify under the same discipline that found the gap.

A Experimental Procedure and Per-Competition Specification Reports

A.1 Per-Competition ML Specification Reports (Attached)

Full specifications for every agent on every competition, written to the five sections of Tso-Jung Yen’s ML specification framework (`ml_pipeline_modules`) — Data / Models & Architecture / Training / Inference / Evaluation & Benchmarking — per competition: 15 reports for my-agent ([kaggle/docs/ml_specs/](https://kaggle.com/docs/ml_specs/), MD + PDF) and 17 for AIDE ([aideml-runs/docs/ml_specs_aide/](https://aideml-runs.com/docs/ml_specs_aide/), MD + PDF).

Every number in these reports is extracted deterministically by `collect.py/collect_aide_facts.py` from that competition’s structured records (`facts.json`, `eda_summary.json`, `experiments_tree_v3.json`, `journal.json`), never generated by an LLM; unavailable fields are marked “not recorded” rather than left blank or estimated. The my-agent report set additionally ships `RUBRIC.md`, an 8-item plan-alignment checklist verified per competition (all 15 pass 8/8). The reports are CV-only by construction; their conclusions must not be mixed with leaderboard verdicts.

This appendix does not repeat per-competition content; below are only the cross-competition rules and the execution environment that individual reports cannot cover.

A.2 Cross-Competition Rule: Split Strategy Is Determined by Task Type

This is the single highest-impact design item in the study, visible only through cross-competition comparison (Table 8).

Table 8: Validation-split policy by task type.

Task type	Required split	Rationale
Time-series extrapolation (test set later than training period)	Time-based split (<code>year < y</code> vs <code>year == y</code>) or <code>TimeSeriesSplit</code>	Each validation block must be strictly later than its training window, mirroring the true train–test gap
Text pairing (same anchor in many rows)	<code>GroupKFold</code> by anchor	Otherwise one anchor spans train and validation, and the model memorizes anchors instead of semantic relations
Generic tabular (continuous i.i.d. target)	Shuffled <code>KFold</code> / <code>StratifiedKFold</code>	No temporal or group structure

Shuffled `KFold` on time-series data is strictly forbidden: on s3e19 this one error cost AIDE a 4.5× optimism bias, while my-agent quarantined the same split as diagnostic (Section 2.2).

A.3 Cross-Competition Rule: DL-to-GBDT Field Mapping

The specification framework contains several deep-learning-specific fields; this study is GBDT-centric, and all per-competition reports follow the mapping in Table 9.

Table 9: Mapping of deep-learning framework fields to their GBDT counterparts.

Framework field	GBDT counterpart
Learning Rate	Boosting shrinkage (<code>learning_rate</code>)
Learning Rate Scheduler	N/A — convergence governed by CV early stopping, not a schedule
Batch Size	N/A — full-dataset histogram boosting, not mini-batches
Model Complexity	Expressed as “number of trees × leaves,” not dense parameter count
Transfer Learning	N/A (no pretrained weights); analogue: prior injection from the experience library
Data Augmentation	N/A (tabular); analogue: feature engineering and seed bagging

A.4 Execution Environment and Reproducibility

Isolation and lane discipline are covered in Section 3.1; Table 10 records only the environment items not stated there.

Table 10: Execution environment.

Item	Detail
Hardware	ARM64 Linux (Ubuntu 24.04), NVIDIA GB10 GPU
Environment management	<code>uv + uv.lock</code> (frozen versions); Python 3.13
Monitoring	<code>bench-watchdog.timer</code> , checking every 15 minutes for (a) machine idle with work pending, (b) process alive but no output for 30 minutes, (c) individual driver death
Determinism	Fixed seeds; <code>LightGBM</code> with <code>deterministic=true</code> , <code>force_row_wise=true</code> , fixed <code>num_threads</code> — necessary for cross-process reproducibility
Per-step memory cap	32 GB (exceeding it marks the node buggy; a single step once consumed 76 GB and stalled the machine)
Output isolation	<code>my-agent kaggle/competitions/</code> ; AIDE <code>aideml-runs/</code> ; NVIDIA <code>nvidia-kaggle-runs/</code>

A.5 Companion Documents

Two companion documents carry material that answers different questions than this report does, each written for a different reader. *Building the Injection Layer: An Engineering Log* asks what broke while building the machinery, for someone rebuilding a similar system. *Per-Competition Case Studies* carries the single-competition forensic depth — the CV–leaderboard reversal mechanics, AIDE’s step-by-step split switch, the two experience-library entries behind the midterm correction, the s5e1 setting behind the pre-registered prediction, and the s3e19 residual decomposition in full — for a reader who wants the evidence behind the condensed claims of Sections 2.2 and 5.2.

The engineering log contains: the per-regime coverage tables behind the consumer-trace finding of Section 5.2, with the audit scripts that reproduce them; the isolation mechanisms summarised in Table 1, together with the record of every occasion one of them failed, what it cost, and how it was caught; the minute-by-minute reconstruction of the voided ventilator attempt; and the nine defects found in a single day’s audit of the injection layer, six from the audit itself and three from verifying the fixes for the first six.

They share a document because they share a failure shape: a produced artifact with no consumer is indistinguishable from a working one — the transferable claim of Section 5.2; the log is the evidence behind it.

A.6 Tree-Search Size per Competition (my-agent)

Table 11: Tree-search node counts per competition (my-agent). The budget machine’s default cap is 60 with per-competition overrides (afsis 80); plateaus stop several runs below 25. tps-sep-2022’s lane finished on the linear protocol without entering tree search.

Competition	Nodes	Competition	Nodes	Competition	Nodes
s3e1	22	s3e14	62	s5e10	50
s3e3	22	s3e16	24	s6e1	48
s3e5	23	s3e19	22	s6e2	60
s3e7	63	s4e11	60	afsis	80
s3e9	26	cat-in-the-dat	65	conway	25
s3e11	24	s5e1	60	tps-sep-2022	—
tps-aug-2022	60	tps-jan-2022	60		

B Code and Skill Packaging

All my-agent code and benchmark artifacts are archived in the project repository github.com/930731haohao-afk/kaggle-agent (private; access granted on request). File paths in the tables below link to the corresponding files in that repository; ai_agents/-level infrastructure scripts are mirrored under [benchmark_infra/](#), and cross-agent result artifacts under [benchmark_results/](#).

B.1 my-agent Skill Definition

Table 12: Skill definition files.

File	Purpose
<code>kaggle/.claude/skills/kaggle-agent/SKILL.md</code>	Main skill definition: six-stage pipeline, tree-search switching condition, experience-library query timing
<code>.../references/01_setup.md – 06_submission.md</code> <code>.../references/07_tree_search.md</code>	Detailed per-stage instructions Tree-search protocol (preferred optimization mode)
<code>kaggle/tree_search/harness_v3.py</code>	Tree-search harness
<code>kaggle/knowledge/experience.md</code>	Cross-competition experience library (every entry carries an evidence field)
<code>kaggle/knowledge/idea_bank.md</code>	Pool of unvalidated ideas

B.2 Experiment Execution Scripts

Table 13: Experiment execution scripts.

File	Purpose
<code>ai_agents/bench-comps/build_isolated_roots.py</code>	Build manifest-validated isolated data roots; officialness judged via the Kaggle API, incl. members of official archives
<code>ai_agents/lane_lock.sh</code>	Lane mutex (atomic directory creation, reclamation on holder death, bounded waiting)
<code>ai_agents/bench_watchdog.sh</code>	Dual-signal monitoring: machine level (alive + advancing) and per driver
<code>ai_agents/kaggle/run_myagent_headless.sh</code>	Unattended my-agent execution (one headless session per competition)
<code>ai_agents/aideml/run_comp.py</code>	Single-competition AIDE run (supports AIDE_COMP_ROOT)
<code>ai_agents/aideml/rerun_20steps.py</code>	Batch AIDE runs (20-step budget)
<code>ai_agents/aideml/collect_aide_facts.py</code>	Deterministic fact extraction from AIDE journals
<code>ai_agents/nvidia-kaggle-runs/run_ready_reproductions.py</code>	NVIDIA reproduction runner (only scripts passing smoke tests)

B.3 Data and Results

Table 14: Data and result artifacts.

Item	Path
Three-way score master table	ai_agents/aideml-runs/three_way_scores.csv
Full three-way comparison report	ai_agents/aideml-runs/THREE_WAY_REPORT.md (with PDF)
AIDE per-competition ML spec reports (17)	ai_agents/aideml-runs/docs/ml_specs_aide/
my-agent per-competition ML spec reports (15)	ai_agents/kaggle/docs/ml_specs/
Coverage status table	ai_agents/BENCHMARK_COVERAGE.md
Benchmark-run manifest	kaggle/docs/rerun_manifest.json
Firing-class lane log (injection-study three-way runs)	benchmark_results/FIRING_CLASS_LANES.md
Per-competition experiment logs	kaggle/competitions/<comp>/experiments.json , experiments_tree_v3.json

B.4 Injection-Layer Components

Table 15: Injection-layer components (all in the project repository).

File	Purpose
<code>.../references/00_problem_dossier.md</code>	Stage 0.5 instructions: dossier schema, typed-operator requirement, forbidden inputs, downstream consumption
<code>knowledge/task_priors.md</code>	Evidence-backed task-level priors ([TASK-*) + external-data whitelist with leakage rules
<code>knowledge/injection_operators.md</code>	The contract between the judgment and execution layers: 10 typed operators, parameter schemas, the evidence for each, and the conditions under which each must not be used
<code>external_data/sources.py</code>	Whitelisted fetchers (11 World Bank indicators, ECB FX, OWID COVID, holidays, ISO), snapshot-recorded atomic caches
<code>external_data/join.py</code>	Leakage-guarded year/month/day joins; declared time units; row-order preservation; audit logs
<code>external_data/apply.py</code>	Operator dispatcher: realizes data-level operators, emits config for config-only ones, writes the coverage ledger, runs the proportionality and covariate-volatility preconditions
<code>external_data/validate_dossier.py</code>	Emission-side conformance check (schema, whitelist, provenance); non-zero exit gates a run
<code>external_data/coverage_audit.py</code>	Computes the coverage split behind Section 5.2 (tables in the engineering log)
<code>external_data/selftest.py</code>	14-section execution-side test suite incl. direct unit test of the leakage invariant
<code>tree_search/make_v5_arm.py</code>	Competition-agnostic arm builder: digit-exact variant workspaces + generated evaluators
<code>tree_search/run_arm_search.py</code>	Variant-lane race driver (harness_v3 budget machine)
<code>tree_search/holdout_year_probe.py</code>	Held-out-year form-selection probe (the negative result of Section 5.2)
<code>tree_search/structural_probe.py</code>	Structural-decomposition probe: tests whether the s3e19 residual is a shape error a GBDT cannot express
<code>tree_search/eval_support.py</code>	Per-fold target encoding, sparse-linear family, runtime-attestation writer (2026-07-31)
<code>tree_search/seed_from_ledger.py</code>	Translates emitted node configs into real seed nodes; writes <code>injection_consumed.json</code> (the consumer trace)
<code>external_data/verify_advisory.py</code>	Advisory verification: static parameter check upgraded by runtime attestation (<code>evaluator_attestation.json</code>)
<code>docs/preregistrations/horizon_length_form_selector.md</code>	H-HORIZON pre-registration: binding protocol on the next firing-class competition
<code>competitions/*/dossier.json</code>	20 dossiers from the sweep (Section 5.1)

B.5 Minimal Steps to Reproduce This Study

0) All paths repo-relative (the `ai_agents/-level` originals map per the mirror
note at the top of this section)

```

# 1) Build isolated data roots (per official Kaggle manifest)
python benchmark_infra/build_isolated_roots.py --write <comp>

# 2) my-agent, one competition
cd ai_agents/kaggle && claude -p "<see run_myagent_headless.sh PROMPT>" \
    --dangerously-skip-permissions

# 3) AIDE, one competition (20 steps)
AIDE_COMP_ROOT=<data-root> \
    python benchmark_infra/run_comp.py <comp> --shim --steps 20

# 4) NVIDIA, one competition
# (generates and runs per-competition reproduce scripts)
python benchmark_infra/run_ready_reproductions.py

# 5) Submit and retrieve public/private scores
KAGGLE_API_TOKEN=$(cat ~/.kaggle/huang_token) \
    kaggle competitions submit -c <comp> \
    -f submission.csv -m "<msg>"

```

Environment: ARM64 Linux (Ubuntu 24.04), NVIDIA GB10 GPU, uv-managed Python 3.13, versions frozen in `uv.lock`.

Every number in this report is traceable to the structured records listed above. This report will be updated when the leaderboard scoring step runs and as the items in Section 7.5 complete.

References

- [1] Aygün, E., et al. An AI system to help scientists write expert-level empirical software. *Nature* (2026). <https://doi.org/10.1038/s41586-026-10658-6>
- [2] Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-Driven Exploration in the Space of Code. arXiv:2502.13138 (2025).
- [3] NVIDIA. Kaggle Agent: kernel-reproduction agent skill (software), 2026.
- [4] Chan, J. S., et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. arXiv:2410.07095 (2024).